

PHÂN TÍCH CHƯƠNG TRÌNH MÔ PHỎNG CÁNH TAY ROBOT 4 BẬC TỰ DO

Author: KienBui_github.com/kienbt0605

1. Giới thiệu

Chương trình được xây dựng bằng Python nhằm mô phỏng chuyển động của cánh tay robot khớp nối 4 bậc tự do (4-DOF Articulated Robot Arm) trong không gian 3 chiều. Mô hình gồm 4 khớp: khớp quay đế J1, khớp vai J2, khớp khuỷu J3 và khớp cổ tay J4. Chương trình không chỉ hiển thị hình học 3D của robot mà còn cho phép người dùng điều khiển góc khớp, nhập điểm đích, và cho đầu công tác di chuyển theo đường thẳng hoặc quỹ đạo tròn.

Về mặt kỹ thuật, chương trình kết hợp các thư viện numpy, math, tkinter và matplotlib. Trong đó, tkinter dùng để xây dựng giao diện người dùng, matplotlib phụ trách hiển thị mô hình 3D, còn numpy được dùng cho các phép toán ma trận và vector trong bài toán động học.

2. Mục tiêu của chương trình

Chương trình được xây dựng với các mục tiêu chính sau:

- Mô phỏng trực quan cấu trúc của robot 4 bậc tự do trong hệ tọa độ 3D.
- Tính toán động học thuận để xác định vị trí đầu công tác từ các góc khớp.
- Tính toán động học nghịch để tìm bộ góc khớp giúp robot đạt tới một điểm đích trong không gian.
- Cho phép mô phỏng chuyển động theo quỹ đạo thẳng và quỹ đạo tròn.
- Hỗ trợ quan sát trực quan quỹ đạo, tọa độ đầu công tác, sai số IK và góc nhìn camera.

3. Cấu trúc tổng quát của chương trình

Chương trình gồm 2 phần chính:

Phần thứ nhất là khối tính toán động học, bao gồm các hàm:

- `dh_matrix()`
- `forward_kinematics_3d()`
- `jacobian_3d()`
- `inverse_kinematics_3d()`

Phần thứ hai là khối giao diện và mô phỏng, được đóng gói trong lớp RobotArm3DSimulator. Lớp này đảm nhiệm việc tạo giao diện, hiển thị mô hình 3D, nhận dữ liệu từ người dùng và điều khiển quá trình mô phỏng quỹ đạo.

4. Các thông số chính của robot

Trong chương trình, robot được khai báo với chiều dài các khâu:

- $LINK_LENGTHS = [90, 90, 30]$
- $BASE_HEIGHT = 75$
- Giới hạn góc các khớp:
 - J1: từ -180° đến 180°
 - J2: từ -90° đến 135°
 - J3: từ -135° đến 135°
 - J4: từ -180° đến 180°

Các thông số này cho biết robot có 3 đoạn tay chính với tổng chiều dài hình học là 210 mm, đặt trên một đế cao 75 mm. Ngoài ra, chương trình còn giới hạn góc quay từng khớp để tránh nghiệm động học nghịch vượt quá miền làm việc thực tế.

5. Phân tích phân động học thuận

5.1. Hàm ma trận DH

Hàm `dh_matrix(theta, d, a, alpha)` tạo ra ma trận biến đổi thuần nhất 4×4 theo quy ước Denavit–Hartenberg. Đây là công cụ nền tảng để mô tả mối liên hệ vị trí và định hướng giữa các khâu liên tiếp của robot.

Ma trận có dạng tổng quát:

$$T = \begin{bmatrix} \cos \theta & -\sin \theta \cos \alpha & \sin \theta \sin \alpha & a \cos \theta \\ \sin \theta & \cos \theta \cos \alpha & -\cos \theta \sin \alpha & a \sin \theta \\ 0 & \sin \alpha & \cos \alpha & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Trong code, các giá trị lượng giác được tính bằng `math.cos()` và `math.sin()` để xây dựng trực tiếp ma trận này.

5.2. Hàm động học thuận 3D

Hàm `forward_kinematics_3d(thetas_deg, lengths=LINK_LENGTHS)` nhận vào 4 góc khớp theo đơn vị độ, sau đó chuyển sang radian, thiết lập bảng tham số DH và lần lượt

nhân các ma trận để suy ra vị trí từng khớp và đầu công tác. Hàm trả về 4 điểm: vị trí gốc tay, đầu khâu 1, đầu khâu 2 và đầu công tác.

Cụ thể, chương trình sử dụng 4 hàng DH:

- Khớp 1: quay quanh trục Z, nâng lên độ cao BASE_HEIGHT, có góc xoắn $\alpha = \pi/2$
- Khớp 2: nối với khâu dài L1
- Khớp 3: nối với khâu dài L2
- Khớp 4: nối với khâu dài L3

Ý nghĩa của hàm này là: khi người dùng chỉnh các slider góc khớp, chương trình dùng động học thuận để tính lại vị trí các điểm đặc trưng của robot và cập nhật hình vẽ 3D ngay lập tức.

6. Phân tích phần động học nghịch

6.1. Jacobian số

Hàm `jacobian_3d()` tính ma trận Jacobian 3x4 bằng phương pháp sai phân hữu hạn. Với mỗi khớp, chương trình tăng góc đó lên một lượng rất nhỏ $\delta = 0.01$ độ, sau đó tính lại vị trí đầu công tác và lấy chênh lệch để xấp xỉ đạo hàm riêng.

Điều này cho phép xác định mức độ ảnh hưởng của từng khớp đến vị trí đầu công tác trong không gian 3 chiều. Vì robot có 4 khớp nhưng đầu ra vị trí chỉ gồm 3 thành phần (x, y, z), Jacobian có kích thước 3x4.

6.2. Thuật toán động học nghịch

Hàm `inverse_kinematics_3d()` giải bài toán động học nghịch bằng phương pháp Damped Least Squares. Quy trình hoạt động như sau:

- Lấy bộ góc hiện tại làm giá trị khởi tạo.
- Tính vị trí đầu công tác hiện tại bằng động học thuận.
- Tính sai số vị trí giữa điểm đích và điểm hiện tại.
- Nếu sai số nhỏ hơn $\text{tol} = 0.005$ thì dừng.
- Nếu chưa đạt, tính Jacobian và cập nhật góc khớp theo công thức gần đúng:

$$\Delta\theta = J^T(JJ^T + \lambda^2 I)^{-1}e$$

Trong đó:

- J là Jacobian

- e là vector sai số vị trí
- $\lambda = 0.8$ là hệ số damping giúp thuật toán ổn định hơn khi gần điểm kỳ dị.

Sau khi tính được $\Delta\theta$, chương trình:

- đổi sang đơn vị độ,
- nhân với hệ số $\alpha = 0.4$ để làm mượt quá trình cập nhật,
- chặn giá trị thay đổi mỗi bước trong khoảng $[-4^\circ, 4^\circ]$,
- ép góc về đúng giới hạn khớp,
- chuẩn hóa góc khớp 1 về miền $[-180^\circ, 180^\circ]$.

Cách làm này phù hợp với mô phỏng thời gian thực vì khá ổn định, dễ cài đặt và không cần phải tìm nghiệm giải tích phức tạp cho robot 4 bậc tự do.

7. Giao diện người dùng

Giao diện được xây dựng trong phương thức `_build_ui()`. Giao diện gồm hai vùng chính:

- **Bên trái:** vùng hiển thị mô hình 3D.
- **Bên phải:** bảng điều khiển gồm slider góc khớp, thông tin đầu công tác, phần nhập mục tiêu đường thẳng, phần nhập mục tiêu quỹ đạo tròn, nhóm nút điều khiển và tùy chọn camera.

Các thành phần đáng chú ý gồm:

- 4 thanh trượt để chỉnh góc khớp.
- Khung hiển thị tọa độ X, Y, Z của đầu công tác.
- Ô nhập mục tiêu đường thẳng (X, Y, Z).
- Ô nhập quỹ đạo tròn (X_c, Y_c, Z_c, R).
- Các nút Reset, Clear Target, Clear Trail.
- Các nút thay đổi góc nhìn Front, Side, Top, Iso.
- Bảng thông số DH ở cuối giao diện.

8. Phần hiển thị mô hình 3D

Phương thức `_draw()` chịu trách nhiệm cập nhật toàn bộ khung nhìn 3D. Hàm này thực hiện các công việc chính:

- Xóa khung vẽ cũ.
- Thiết lập giới hạn trục X, Y, Z.

- Vẽ lưới mặt đất và hệ trục tọa độ.
- Vẽ chân đế robot, đĩa đế và cột nâng.
- Gọi động học thuận để lấy vị trí các khớp.
- Vẽ các khâu bằng đoạn thẳng màu.
- Vẽ các khớp bằng các điểm tròn.
- Vẽ đầu công tác bằng marker hình thoi.
- Lưu và vẽ quỹ đạo chuyển động của đầu công tác.
- Nếu có mục tiêu, hiển thị điểm đích và đoạn nối từ đầu công tác đến mục tiêu.
- Cập nhật các nhãn tọa độ, khoảng cách và sai số IK.

Điểm hay của `_draw()` là không chỉ dùng để hiển thị robot, mà còn trực tiếp phản ánh trạng thái điều khiển, giúp người dùng theo dõi sai số và quỹ đạo di chuyển một cách trực quan.

9. Điều khiển theo quỹ đạo tròn

Phương thức `_solve_ik()` xử lý bài toán cho đầu công tác chuyển động theo **đường tròn**. Người dùng nhập tâm đường tròn (X_c, Y_c, Z_c) và bán kính R .

Quy trình hoạt động như sau:

- Kiểm tra dữ liệu nhập.
- Nếu đang có animation cũ thì hủy.
- Xóa vết quỹ đạo cũ.
- Tính điểm bắt đầu của đường tròn tại góc 0.
- Dùng IK để tìm cấu hình khớp tương ứng với điểm bắt đầu.
- Nội suy mềm từ trạng thái hiện tại sang trạng thái bắt đầu trong 30 bước bằng hàm easing:

$$te = t^2(3 - 2t)$$

- Sau đó cho đầu công tác chạy quanh đường tròn với 100 mẫu điểm.
- Mỗi bước cách nhau 30 ms và đều gọi lại IK để cập nhật góc khớp.

Cách tiếp cận này cho phép quỹ đạo nhìn khá mượt và trực quan, phù hợp mục tiêu mô phỏng.

10. Điều khiển theo đường thẳng

Phương thức `_solve_line_ik()` cho phép đầu công tác di chuyển từ vị trí hiện tại đến điểm đích (X, Y, Z) theo đường thẳng trong không gian.

Thuật toán được thực hiện như sau:

- Lấy tọa độ đầu công tác hiện tại bằng động học thuận.
- Tính khoảng cách từ điểm đầu đến điểm đích.
- Xác định số bước nội suy:

$$n_steps = \max(30, \text{int}(dist/1.5))$$

- Ở mỗi bước, nội suy tuyến tính để tạo ra điểm trung gian (cx, cy, cz).
- Dùng IK để tìm góc khớp cho điểm trung gian đó.
- Cập nhật robot sau mỗi 30 ms.

Ưu điểm của cách này là đầu công tác sẽ bám theo quỹ đạo thẳng tốt hơn so với việc chỉ nhảy trực tiếp tới điểm đích. Tuy nhiên, do robot vẫn giải IK tại từng điểm nội suy, độ mượt thực tế còn phụ thuộc chất lượng hội tụ của thuật toán IK.

11. Các hàm hỗ trợ khác

Ngoài các hàm chính, chương trình còn có một số hàm hỗ trợ:

- `_on_slider()`: cập nhật góc khớp khi người dùng kéo thanh trượt.
- `_reset()`: đưa robot về tư thế ban đầu [0, 60, -30, 0] và xóa vết quỹ đạo.
- `_clear_target()`: xóa mục tiêu hiện tại.
- `_clear_trail()`: xóa quỹ đạo vết.
- `_set_view()`: đổi góc nhìn camera.
- `_animate_ik()`: hiện đang để pass, tức là chưa được triển khai.

12. Đánh giá ưu điểm của chương trình

Chương trình có nhiều ưu điểm:

Thứ nhất, cấu trúc rõ ràng, tách khá hợp lý giữa phần tính toán động học và phần giao diện.

Thứ hai, chương trình có tính trực quan cao nhờ hiển thị mô hình 3D, quỹ đạo vết, điểm mục tiêu và sai số. Điều này rất phù hợp cho mục đích học tập, trình bày hoặc kiểm tra thuật toán động học robot.

Thứ ba, việc dùng Jacobian số và Damped Least Squares giúp bài toán IK dễ mở rộng, không cần tự suy ra công thức động học nghịch giải tích.

Thứ tư, chương trình đã hỗ trợ hai dạng chuyển động quan trọng là chuyển động theo đường thẳng và quỹ đạo tròn, giúp mô hình có giá trị thực hành tốt hơn một phần mềm chỉ điều khiển bằng slider.

13. Hạn chế và điểm chưa đồng nhất

Bên cạnh các ưu điểm, chương trình vẫn có một số hạn chế.

Một điểm chưa đồng nhất khá rõ là chiều dài các khâu trong phần tính toán đang là [90, 90, 30], nhưng phần nhãn giao diện lại hiển thị $L1=120\text{mm}$, $L2=80\text{mm}$, $L3=50\text{mm}$. Điều này dễ gây nhầm lẫn khi người dùng đối chiếu giữa mô hình toán học và giao diện.

Ngoài ra, phần mô tả đầu file ghi rằng “Base tại gốc tọa độ (0,0,0) trên mặt phẳng XY”, nhưng trong động học thuận vị trí ban đầu lại được đặt tại (0,0,BASE_HEIGHT) với $\text{BASE_HEIGHT} = 75$. Nghĩa là về bản chất mô hình đang coi điểm khởi đầu của chuỗi khâu nằm trên đỉnh đế, không phải ngay trên mặt phẳng gốc. Cách mô tả và cách cài đặt chưa hoàn toàn khớp nhau.

Một hạn chế khác là chương trình mới giải vị trí đầu công tác (x, y, z), chưa xét đến định hướng của đầu công tác. Vì vậy robot phù hợp cho mô phỏng vị trí điểm cuối hơn là các bài toán thao tác cần điều khiển pose đầy đủ.

Ngoài ra, Jacobian đang được tính theo sai phân số nên tuy đơn giản nhưng có thể chậm hơn và kém chính xác hơn so với Jacobian giải tích nếu mở rộng mô hình lớn hơn.

14. Hướng phát triển

Để nâng cấp chương trình, có thể bổ sung các hướng sau:

- Đồng bộ lại toàn bộ thông số chiều dài khâu giữa phần tính toán và giao diện.
- Bổ sung điều khiển định hướng đầu công tác.
- Thay Jacobian số bằng Jacobian giải tích để tăng tốc độ.
- Bổ sung kiểm tra miền làm việc trước khi chạy IK.
- Thêm tính năng lưu quỹ đạo, xuất dữ liệu tọa độ hoặc góc khớp.
- Hoàn thiện hàm `_animate_ik()`.
- Kết nối mô hình với robot thật hoặc mô phỏng servo theo thời gian thực.

15. Kết luận

Tổng thể, đây là một chương trình mô phỏng cánh tay robot 4 bậc tự do khá tốt cho mục đích học tập và nghiên cứu cơ bản. Chương trình đã tích hợp được các nội dung cốt lõi của robot học gồm mô hình DH, động học thuận, động học nghịch, hiển thị 3D và mô phỏng quỹ đạo. Dù vẫn còn một số điểm chưa thống nhất về thông số và chưa xử lý định hướng đầu công tác, chương trình vẫn thể hiện rõ tư duy xây dựng một hệ mô phỏng robot hoàn chỉnh, có tính tương tác và trực quan cao.